

What Does Scale Buy in Tool-Use Agents? A Capability×Scale×Lever Map Showing Compliance Is Scale-Invariant

ICLR submission draft — branch `facet-rft-2026`. Numbers tagged [SETTLED] are measured (all with zero paid frontier-API calls; the user-simulator is local); [EST] are estimates/placeholders for experiments not yet run.

Abstract

Tool-use agents (NL request $\rightarrow$ tool calls under a policy) are usually improved by scaling the base model. We instead ask, function by function: *what does scale actually buy, what can a model genuinely not do at any scale, and what is cheaply covered by deterministic scaffold or by per-domain authored relations (A2)?* Treating four benchmarks (τ^2 -bench, SOPBench, TaskBench, Synth) purely as measurement instruments, we decompose tool-use failure into **operand execution**, **orchestration load**, and **compliance/guarantee**, and chart each across model scale (7B–32B measured; 72B–235B [EST]) and across levers (base / scaffold / A2 / learning). Three findings organize the map. (1) **Operand execution saturates early**: given an explicit specification a 32B base is exactly correct 88/88 (100%), so the residual "operand failures" are not a learnable capability gap. (2) **Capability-type load retires with scale in a definite order**: of five candidate load dimensions, interference and state-multiplicity are predictive of failure at 7B/14B but vanish by 32B, while length and conditional-depth persist. (3) **Compliance is scale-invariant**. As scale rises, task-pass climbs (7B 0.19 $\rightarrow$ 32B 0.55) but the *compliant*-pass gap stays flat (~ 0.056) and policy-violations do **not** fall (g2-class 31 $\rightarrow$ 41 across 7B $\rightarrow$ 32B); a deterministic gate eliminates them at every scale. We conclude that scale buys *capability* but not *guarantee*, so even a frontier model needs a deterministic guarantee-layer — and a small base + capability-scaffold (which recedes as scale grows) + a scale-invariant guarantee-scaffold + authored A2 is the cost-efficient way to reach frontier-level tool-use. We frame deterministic scaffold as a **load-reduction operator** and give the function×scale×lever map as the paper's contribution.

1. Introduction

A tool-use agent maps a natural-language request to a sequence of tool calls that must satisfy a domain policy. The default recipe for better agents is a bigger base model. But "bigger" conflates several distinct things, and it is expensive: model size roughly doubles inference OpEx and hardware CapEx. The practical question for on-premise deployment is not "how big a model" but "**which capabilities must scale up, which can be handed to cheap deterministic structure, and which no model — frontier included — can be trusted to do?**"

We answer this with a **function $\times$ scale $\times$ lever** decomposition. For each functional sub-capability of tool-use we ask, across model scale, whether it (a) grows with scale, (b) is a genuine model limit that scale does not fix, (c) is efficiently covered by deterministic *scaffold*, or (d) is covered by per-domain authored relations/policies (A2). The benchmarks are instruments for filling cells of this map, not the object of study.

Contributions. 1. A controlled, **frontier-API-free** methodology that isolates operand / orchestration-load / compliance and measures each across scale (§4). Isolated single-shot probes and existing multi-scale runs avoid the cost and noise of full agent rollouts. 2. **Operand execution is scale-saturated, not learnable** (§5.1): a 32B base is 100% correct given a specification; the apparent gap is criterion-interpretation handled deterministically, not a capability deficit. Faithful-formalize SFT is therefore NO-GO at 32B. 3. A **load decomposition** and its **scale-response** (§5.2): the set of load dimensions that bind is *scale-dependent* — scale retires interference and state load by 32B while length and conditional-depth persist. Controlled probes overturn an observational correlation, demonstrating the necessity of controlled (not merely observational) measurement. 4. The central finding: **compliance/guarantee is scale-invariant** (§5.3). Capability rises with scale but the policy-violation rate of the *compliant*-pass metric does not; a deterministic gate closes it at every scale. This is the empirical backbone of a **two-kind scaffold** thesis — capability-scaffold recedes with scale (F3, pass), guarantee-scaffold is scale-invariant (F4, compliant-pass) — and explains *why even frontier systems need a deterministic guarantee layer*. 5. A unifying frame: **scaffold = a load-reduction operator** (§6), giving the prediction $\text{pass} \Leftrightarrow L_{\text{eff}} < L^*(N)$ with $L_{\text{eff}} = L - \Delta L_{\text{scaffold}}$ and an *independent* estimate of ΔL .

Our headline is deliberately **not** "small beats large": that result is already held by RL-routing systems (§2). Our contribution is *which* parts of the gap are scale-invariant and must be offloaded, and a map that tells a deployer which lever to pull per capability.

2. Related Work

Our contribution sits at the intersection of *verified prior pieces*; the central lineage is conceptual (why an LLM is good at language but must offload deep procedures and cannot self-guarantee compliance), not the recent "small-beats-large" routing systems, which address a narrower problem (§2.5).

§2.1 Why language is shallow-parallel and deep procedures must be offloaded (the core lineage). A bounded-depth Transformer forward pass is, asymptotically, a constant-depth parallel circuit (uniform TC $\square$; Merrill & Sabharwal, arXiv 2207.00729), and chain-of-thought escapes this only by unrolling depth into serial tokens (Feng et al., arXiv 2305.15408 / 2402.12875; RASP-L length-generalization, Zhou et al.). Natural language, meanwhile, *embeds* procedures of definite computational class: **van Benthem's semantic automata** make "every/some" finite-state but "most/less-than-half" provably pushdown, and this hierarchy is cognitively real (working-memory/PFC recruitment and reaction-time scale with the minimal-automaton class). **Relevance Theory** (Blakemore; Wilson) distinguishes *conceptual* from *procedural* meaning (the latter sub-personal, "machine-language"), and **Goodman's notationality** (disjointness + finite differentiation) gives the formal condition under which a symbol system supports unambiguous mechanical manipulation. The representational-effect correction (Zhang & Norman) clarifies that notation sets *cost*, not computability. Together these say: classifying *which* deep procedure a request embeds is shallow (LLM-learnable, transferable), but *executing* it is serial-deep and must be offloaded — the foundation of our capability/lever routing. (Project synthesis: NL_PROCEDURE_OFFLOAD_THEORY; deep-research reports under [deepresearch/](#).)

§2.2 Cognitive architectures, non-introspective decidability, and scale-invariant epistemic limits. Our system is an instance of the **cognitive-architectures-for-language-agents** umbrella (CoALA, Sumers et al., arXiv 2309.02427, TMLR); an LLM as a probabilistic production within a memory/action/decision loop; we make concrete the substrate CoALA leaves general (a relational-algebra deterministic core, an empty-relation abstention, zero-retrain A2-swap). **SOAR** (Newell, Laird & Rosenbloom 1987; Laird 2012; cognitive-design-patterns, Wray, Kirk & Laird, arXiv 2505.07087) surfaces indecision *architecturally* — an *impasse* observed from the absence of a preference, not from introspection — and subgoals; this is the precedent for our "uncertainty $\rightarrow$ observable empty/ambiguous relation $\rightarrow$ ASK" (tie = $\sigma > 1$, no-change = $\sigma = 0$). The *deterministic-first / LLM-fallback* execution pattern is established (Bootstrapping cognitive agents, Zhu & Simmons, arXiv 2403.00810, 50–100 $\times$ fewer reasoning tokens; NL2GenSym, arXiv 2510.09355, LLM $\rightarrow$ SOAR-rule synthesis). We add what this lineage lacks: a **formalize-faithfulness** axis (SOAR assumes productions are correct), a *learned* empty $\rightarrow$ ASK, and a small *weight-learned* domain-general skill with A2-swap transfer. On the epistemic side, **hallucination is provably scale-invariant** (Xu, Jain & Kankanhalli, arXiv 2401.11817; Kalai et al., arXiv 2509.04664: scoring rewards guessing over abstention), and verbalized self-confidence is non-diagnostic **on our exact benchmark** (τ^2 -bench confidence $\approx$ random, AUROC 0.47–0.69, inflating with trajectory length; arXiv 2602.05073 [pilot-scale]). The mis-formalize failure we target (a wrong-but-non-empty formalization that the deterministic checker cannot catch) is now observed in *frontier* models: "Do LLMs Game Formalization?" (Kim et al., arXiv 2604.19459) finds a detection-evading premise-mistranslation mode (a minority but checker-invisible), the direct analogue of our silent $\blacksquare$ residual. Compositional collapse with complexity is documented (Faith and Fate, arXiv 2305.18654; GSM-Symbolic, arXiv 2410.05229; planning collapse, arXiv 2409.13373; correctness-guarantee 0, arXiv 2410.02162). **Our §5.3 result is the policy-enforcement analogue of all of the above:** just as no model self-guarantees truthfulness, none self-guarantees *compliance* — both scale-invariant, both demanding an external deterministic mechanism (abstention/impasse; a gate). (*Honesty, per the source review: faithfulness-gaming is a minority mode — most errors are faithful — and the cited works do not show "more deep $\Rightarrow$ more fabrication"; we do not claim that.*)

§2.3 LLM-proposes / deterministic-executes (offload boundary). The split we inherit is mainstream: LLM-Modulo (Kambhampati et al., arXiv 2402.01817, ICML 2024), PAL (program-aided), and NL $\rightarrow$ PDDL *formalizers* that beat direct planning (e.g. 100-block 100% vs $\sim$ 20%), with RLVR systems whose "V" is a deterministic checker (AlphaGeometry, Nature 625, 2024; AlphaProof 2024; Tülu 3, arXiv 2411.15124). Neurosymbolic motivation: Garcez & Lamb (arXiv 2012.05876), Kautz (AI Mag. 43(1), 2022), Marcus (arXiv 2002.06177). **Honest boundary:** offload does *not* always win — LLM-as-formalizer loses to direct solving on simple cases (formalization overhead) and can leak solution reasoning into the spec; the boundary is itself an object of measurement (our $d(e)$ / bounded-budget probes).

§2.4 Guarantees / shielding. Runtime enforcement from safe-RL shielding (Alshiekh et al., AAAI 2018; Jansen et al., arXiv 1807.06096) and predictive safety filters (Wabersich & Zeilinger, Automatica 2021) to LLM-agent instances: ShieldAgent (arXiv 2503.22738, probabilistic rule circuits, precision <1), AgentSpec (arXiv 2503.18666, hand-written DSL), Formal-LLM (arXiv 2402.00798). These supply a *blocking* safety layer with hand-authored specs or probabilistic checks; our gate is deterministic,

drives (repairs) preconditions rather than only vetoing, and is paired with a learned proposer and zero-retrain transfer.

§2.5 Learned routing and "small-model orchestration" (the narrow rivals). Classification/routing of LLM behaviour is a learned, transferable skill — RTR, A2FM, xRouter, ARM2, ToolkenGPT, and When2Call (tool-necessity linearly decodable from hidden state, AUROC 0.89–0.96; RPO■SFT); RITE shows math-only training transferring cross-domain — while *verbalized self-confidence is distrusted* (criterion-placement, not metacognition), reinforcing external/learned routing over introspection. **ToolOrchestra (arXiv 2511.21689) belongs here and is narrower than it first appears:** it is an 8B model trained by RL to decide *when to delegate to a frontier model vs. answer locally* — i.e. learned **cost-routing of frontier calls** over black-box components (reported 80.2%@10.3¢ vs GPT-5 77.7%@31.3¢ on τ^2). It optimizes *cost given* the components; it does **not** provide deterministic offload, by-construction compliance, decidable-vs-learned accounting, or zero-retrain domain transfer, and it *relies on frontier delegation* — disqualifying for the on-prem / no-frontier-call / no-data-export regime. It establishes that a cost-favorable small-model result *exists*, which is exactly why our contribution is the *analysis and method* (what is scale-invariant, which lever per capability), not that result. ATA (arXiv 2510.16381, FOL+Z3, training-free) is per-domain with no transfer.

§2.6 Load, long-context, and benchmarks. Our load framing defers to cognitive-load theory and long-context "lost-in-the-middle" degradation; novelty is only *load-decomposition* $\times$ *scaffold-load-reduction for tool-use*, not a theory of load. Benchmarks as instruments: τ^2 -bench (arXiv 2406.12045), SOPBench (arXiv 2503.08669), TaskBench, Synth/CFB (arXiv 2501.10132); the floor-measurement precedent separating info-limited vs reasoning-limited failure is "Sufficient Context" (Joren et al., ICLR 2025).

Whitespace. Prior work supplies each piece — procedural-semantics/automata (what is deep), Transformer-expressivity (why a forward pass cannot execute it), SOAR/impasse and CoALA (the cognitive-architecture umbrella and non-introspective surfacing of indecision), hallucination-inevitability (scale-invariant epistemic limit), formalize-offload (LLM proposes/engine executes), learned routing (classification transfers), shielding (deterministic enforcement). The unoccupied intersection *this paper* occupies is the **empirical function $\times$ scale $\times$ lever map**: *which* capabilities scale retires, *which* are genuine scale-invariant limits (compliance, measured), and *which* lever — capability-scaffold (recedes), guarantee-scaffold (invariant), A2, or learning — covers each. The recent routing systems (ToolOrchestra, TRUST) optimize cost over black boxes; we open the box. (*Portfolio note: the "closure-justified finite generator basis + learned rule-abstraction transfer" claim of our prior related-work synthesis — once our intended headline — is, after the operand make-or-break verdict, re-assigned to the companion papers: A2 generation (Paper 2), path-selection heuristics (Paper 3), and the cost-optimal system (Paper 4); the present paper claims the scale-invariance map, not the transfer mechanism, which prior work [schema-guided DST, ToolLLM] already established as a variant.*)

3. The Capability $\times$ Scale $\times$ Lever Framework

We decompose a tool-use trajectory into typed functional steps and, for each function f , populate a row of a map indexed by scale N and lever $\blacksquare \in \{\text{base, capability-scaffold, guarantee-scaffold, A2, learning}\}$. Each cell records: does capability on f grow with N ; is f a genuine model limit; does a lever cover f efficiently.

Three structural claims organize the map:

- **Operand vs orchestration vs compliance.** Operand = atomic execution given a criterion (pick the variant matching a spec). Orchestration = holding multiple interdependent steps coherent under *load*. Compliance = obeying the domain policy with a *guarantee*. These fail for different reasons and respond to different levers.
- **Decidability-first lever routing.** A function that is exactly specifiable (resolution, eligibility, aggregation, provenance) is handed to deterministic scaffold; domain-specific facts are *provided* via A2/retrieval; only domain-general translation/reasoning is *learned*. This inverts the default "scale it" reflex.
- **Two kinds of scaffold.** *Capability-scaffold* (present/compute/resolve) substitutes for a skill the small model lacks and therefore **recedes as scale grows** (F3, raw pass). *Guarantee-scaffold* (policy gates) provides a property no probabilistic model gives and is therefore **scale-invariant** (F4, compliant-pass). §5.3 measures the split.

Load as a vector. Orchestration "load" is not scalar. From failure forensics we operationalize five computable candidate dimensions: **L_len** (context to retain), **L_state** (interdependent state carried), **L_branch** (conditional-resolution depth), **L_interf** (confusable similar entities), **L_contra** (mid-stream contradiction/revision). Crucially, **load $\perp$ operand difficulty**: load can be raised while operand difficulty is held fixed.

Scaffold as a load-reduction operator. Each scaffold component removes a specific load dimension: present/autofetch externalizes reads ($\downarrow L_len$, $\downarrow L_interf$); compute offloads aggregation ($\downarrow L_state$); gates enforce eligibility ($\downarrow L_branch$); a controller holds state ($\downarrow L_state$, $\downarrow L_len$). This gives an *effective* load $L_eff = L - \Delta L_scaffold$ and the prediction

$$pass(f, N) \Leftrightarrow L_eff(f) < L^*(N),$$

where $L^*(N)$ is the model's load-tolerance at scale N . To avoid tautology, ΔL must be estimated *independently* (from the mechanical feature-reduction the scaffold performs) and then shown to predict the failure-onset shift — not read off that shift. The full predictive test requires the controlled curves of §5.2 at more scales **[EST]**; here we establish the components.

4. Method

Benches as instruments. We measure on τ^2 -bench retail (primary), with SOPBench/TaskBench/Synth as the canonical *training* substrate for the lever-allocation that Papers 2–4 build on; τ^2 is a *transfer* target, never trained on. Scale points: Qwen2.5 7B / 14B / 32B-GPTQ-Int8 **[SETTLED]**; 72B / 235B **[EST]**.

Cost discipline (frontier-API-free). The only paid component would be a GPT-4.1 user-simulator; we avoid it. Capability is measured with **isolated single-shot probes** (operand and load probes that pose one decision with a fixed specification) and with **existing multi-scale agent runs** already on disk. A local user-simulator (scripted faithful turns over the local base model) replaces the paid simulator for full-flow checks. All results are persisted under `sim_results/`.

Conditions. *floor* (no scaffold) isolates the base model; *g15 / present / nested / assembled* add capability- and guarantee-scaffold; *GIVEN-SPEC* vs *GOAL* controls the user-simulator input to separate operand execution from criterion interpretation.

Metrics. Robust $pass^{all}$ (all trials) over per-trial point estimates; **F3** = raw task pass (bench/strict); **F4** = compliant pass (full, deducting policy violations); per-dimension **partial correlation** of failure with each load feature, controlling operand difficulty; controlled **accuracy-vs-load** onset curves. We do *not* report single-trial point estimates as findings (user-sim noise ≈ 0.11).

5. Results

5.1 Operand execution is scale-saturated, not a learnable gap **[SETTLED]**

A controlled experiment varies only the user-simulator input. Given an explicit option specification (**GIVEN-SPEC**), the 32B base selects the correct variant **88/88 (100%)**. Given only the goal (**GOAL**), it is 62/88 (70%); the 30% gap is criterion interpretation ($\argmax/\argmin$ = deterministic compute, multi-attribute reasoning, conversational fidelity), not execution. Genuine join/disambiguation (user describes, does not name, the order) is 7/13 (54%) and is present-addressable. **Implication:** operand is not a capability gap at 32B, and faithful-formalize SFT is **NO-GO** — there is no residual to learn. (Forensic note: every prior "operand failure" we chased dissolved into a measurement artifact — a calc bug, a premature single case, an ID-given probe artifact — until the controlled probe read 100%.)

The complementary limit is that *selection over a large candidate set does not scale in-head*: on a depth-scale probe, *comparative* over 50 candidates scores **0.02** and $rank@50 \approx 0.30$ even including a 235B model, while the deterministic engine is trivial (1.00) **[SETTLED]**. So "pick the best/most among many" is not done by raw abstraction at any scale; it is offloaded. Operand thus splits cleanly: *atomic execution given a spec* is saturated (base, 100%), and *large-N selection* is a deterministic-compute lever — neither is a learning target.

5.2 Capability-type load retires with scale, in order **[SETTLED 7B–32B]**

Across the five candidate load dimensions, partial correlation of *floor* failure with each feature (controlling operand difficulty) gives:

dimension	7B	14B	32B
L_len (length)	+0.20	+0.33	+0.37
L_state (state)	+0.24	+0.21	+0.06
L_branch (conditional)	+0.20	+0.13	+0.26

L_interf (interference)	+0.30	+0.20	+0.08
L_contra (revision)	+0.06	+0.13	+0.04

The **binding set is scale-dependent**: at 7B/14B four dimensions predict failure; by 32B only length and conditional-depth survive. Scale *retires* interference and state-multiplicity first; length and conditional-depth persist. L_contra is too sparse in τ^2 to fit. This is the precise empirical form of "scale buys capability": it removes load dimensions in a definite order.

Controlled > observational. The above is observational and confounded with task size. A *controlled* interference probe (fix operand; vary only the number N of confusable variants) does **not** reproduce the strong 7B interference signal:

N (distractors)	1	2	4	8	16
32B	1.00	1.00	1.00	0.97	0.87
7B	1.00	1.00	1.00	0.90	0.80

The 7B–32B gap is small (0.80 vs 0.87 at N=16); the observational L_interf correlation was largely a size confound. This is the methodological payoff of controlled generation, and a caution: single-shot probes under-test the inherently *multi-turn* dimensions (L_state, L_contra) [EST: multi-turn controlled probes pending].

The residual is orchestration, not operand. Forensically re-running the robust fail-all set under faithful (local, gpt-4.1-free) turns: 2/10 are user-simulator-noise flips, while the dominant residual is *orchestration-under-load* — multi-item batching that violates a non-pending precondition, multi-order tracking that drops an order and fabricates a placeholder id, conditional-branch sequencing, and context-window overflow. An isolation **plan probe** (elicit the abstract action plan with reads pre-supplied, grade structure only) shows the core action structure is planned correctly in 6/10 cases (e.g. the batch the live run split is batched correctly in isolation), i.e. roughly half the orchestration residual is *execution-load* (closable by deterministic plan/execute separation) and half is genuine in-isolation planning miss (conditional / multi-order grounding) — the irreducible piece that motivates a learned path-selection lever (companion Paper 3), not a bigger base. [SETTLED, forensic; the assembled present+nested+gate stack ceiling is robust pass^{all} = 0.402.]

5.3 Compliance is scale-invariant — the central finding [SETTLED 7B–32B]

We separate **F3** (task pass) from **F4** (compliant pass; passing trajectories that also commit no policy violation). On *floor* (no gate):

scale	F3 (pass)	F4 (compliant)	gap	violations (g1 / g2)	total viol%
7B	0.189	0.130	0.058	57 / 31	33.6%
14B	0.468	0.404	0.064	38 / 38	26.3%
32B	0.547	0.491	0.056	6 / 41	14.3%

Capability (F3) climbs steeply with scale, but **the F3–F4 gap is flat (~0.056) and the policy-violation class g2 does not fall (31→41) — it slightly rises** even as the auth-type class g1 collapses (57→6). With the deterministic gate (g15) the picture is uniform across scale:

scale	F3	F4	gap	viol%
7B	0.162	0.162	0.000	0.0%
14B	0.469	0.469	0.000	0.0%
32B	0.547	0.547	0.000	0.0%

Interpretation. Scale solves *capability-type* violations (g1, auth) but **not policy-semantic compliance** (g2): the compliant-pass gap is scale-invariant. The gate zeros it at every scale, including the most capable. Therefore a probabilistic model — frontier included — cannot be trusted to *guarantee* policy compliance, and a deterministic guarantee-scaffold is needed at all scales. This is the policy-enforcement analogue of the scale-invariance of hallucination (§2), and it is the empirical backbone of the two-kind

scaffold (F3 recedes / F4 invariant). (Caveat: range is 7B–32B; the gated gap=0 holds by construction, so the informative quantity is the non-closing floor gap; the frontier projection rests on the flat trend plus the hallucination-inevitability priors, not on a >72B measurement.)

5.3b Cheap-replication: a capability-scaffold substitutes for scale [SETTLED]

Raw task-pass rises monotonically with scale (pass¹ = 7B 0.24 / 14B 0.52 / 32B 0.60, n=342) — the headline "scale buys capability" curve, consistent with the floor F3 trend of §5.3. The point of the map is that *each piece scale buys can be obtained more cheaply than scale*. A direct instance: a deterministic **fetch-first** engine (provenance-deny → deterministic producer call → inject the real value) moves grounding errors 33 → 9 and roughly **doubles pass (0.14 → 0.264) with zero learning** — a capability-scaffold standing in for scale. On the lever side, not every scaffold helps: an eligibility-steering gate (G5) has ~zero causal effect (the model does not learn to use the guidance) and a naive retry is counter-productive, while the auth/confirm/ownership/precondition gates (G1–G4) are genuine levers. These results populate the "capability-scaffold recedes / guarantee-scaffold invariant" split with measured cells, and ground the cheap-replication thesis of the companion system paper.

5.4 The map, filled

function	scale-response	genuine limit?	efficient lever
operand execution	saturates by 32B (100% given-spec)	no	base (no learning needed)
interference / state load	retire by 32B	no	capability-scaffold (present), or scale
length / conditional load	persist at 32B	partial	capability-scaffold (partial); conditional → controller/learn
policy compliance (g2)	scale-invariant	yes (guarantee)	guarantee-scaffold (gate)
domain facts	n/a	n/a	A2 / retrieval

6. Discussion

The map yields a deployment rule that inverts "scale it": **reduce the load, don't train the model to tolerate it**. Capability-scaffold removes the load dimensions scale would otherwise have to buy (and thus recedes when scale is available); guarantee-scaffold supplies the property scale never buys (and is thus kept at every scale, frontier included). Where a load is irreducible by scaffold — conditional-depth / path-search is our candidate — the lever shifts to a *learned, transferable* heuristic (Paper 3), not to a bigger model. Training the model to tolerate load is, in our framing, training it to do the deterministic controller's job; the two-wing thesis routes that work to determinism by default and to learning only for the residual.

7. Limitations

- **Scale range.** Measured 7B–32B; 72B/235B and true frontier are [EST]. The scale-invariance claim is a flat-trend projection plus priors, not a >72B measurement.
- **Observational vs controlled.** §5.2's correlations are confounded with task size; only the controlled probe (one dimension done, L_interf) is causal. L_branch/L_len controlled curves and multi-turn (L_state/L_contra) probes are [EST].
- **Single bench for compliance.** F3/F4 measured on τ^2 retail; cross-domain (airline/bank) and cross-bench (SOPBench/TaskBench/Synth) replication is [EST].
- **Transfer.** Zero-retrain ABox-swap is the design premise; its cross-field validation is Paper 4 and is [EST/partial] here (SOPBench cross-domain is partially measured).
- **Construction artifact.** The gated gap=0 is by construction; we rely on the *floor* gap's non-closure for the substantive claim.

8. Conclusion

Scaling a tool-use agent buys capability — it retires load dimensions in a definite order and lifts task pass — but it does **not** buy compliance: policy-violation rates are flat across 7B→32B while pass climbs, and a deterministic gate removes them at every scale. The right object is therefore not the model size but the **function×scale×lever map**: a small base, a capability-scaffold that recedes as scale grows, a scale-invariant guarantee-scaffold, and authored A2 — with learning reserved for the domain-general residual. This is why even frontier systems should use deterministic scaffold and A2, and it sets up the system that optimizes the lever mix by cost (Paper 4), the methods that generate A2 (Paper 2), and the learned heuristics for the irreducible path-search residual (Paper 3).

The complete bibliography (229 entries, organized by theme, deduplicated across the project's deep-research and literature-review corpus) follows below. Entries flagged [unverified] / future-dated are to be re-checked before submission.

Paper 1 — References (229 total, deduped across deep-research + litreview/relwork sources)

Collapsed 7 cross-source duplicate pairs (same arXiv id), merging the relationship notes of each: LATS (2310.04406), RAP (2305.14992), ReST-MCTS* (2406.03816), Math-Shepherd (2312.08935), PAL (2211.10435), ToolLLM (2307.16789), Voyager (2305.16291). Starting totals 89 (deep-research) + 147 (litreview) = 236; after dedup = 229.

1. Foundations — transformer expressivity & bounded-depth, procedural vs conceptual semantics, semantic automata, notationality & representation-cost

1. Kim & Suzuki (2024). *Transformers need intermediate-step supervision to learn parity in one step (ICLR25 Oral)*. arXiv:2410.08633. — [transformer-expressivity] intermediate-step loss yields proven efficiency separation; grounds derivation supervision.
2. Feng et al. (2023). *Towards Revealing the Mystery behind Chain of Thought*. arXiv:2305.15408 (NeurIPS23). — [transformer-expressivity] no-CoT const-depth = AC0/TC0; CoT evaluates size-T boolean circuit.
3. Feng et al. (2024). *Chain-of-Thought expressivity (follow-up)*. arXiv:2402.12875 (ICLR24). — [transformer-expressivity] CoT escapes TC0; supports inductive intermediate-emit prescription.
4. Merrill & Sabharwal (2022). *The Parallelism Tradeoff: Limits of Log-Precision Transformers*. arXiv:2207.00729. — [transformer-expressivity] log-precision forward pass = uniform TC0; bounded serial depth.
5. Bhattamishra et al. (2022). *On the difficulty of learning Boolean functions / low-sensitivity bias*. arXiv:2211.12316 (ACL23). — [transformer-expressivity] flat-AND easy, parity collapses; ICL accuracy vs boolean complexity $r=-0.88$.
6. Wang et al. (2024). *Boolean-function sensitivity in LLMs*. arXiv:2412.02823. — [transformer-expressivity] low-sensitivity bias corroborates flat-AND vs high-sensitivity collapse.
7. Abbe et al. (2024). *Globality barrier and inductive scratchpad*. arXiv:2406.06467 (NeurIPS24). — [transformer-expressivity] globality barrier; only inductive/structured scratchpad breaks it.
8. He et al. (2025). *Looped Locate-and-Replace (bottom-up reduction)*. arXiv:2512.02677. — [transformer-expressivity] bottom-up reduction mitigates OOD depth-decay; inference-time loop. [unverified — future-dated preprint]
9. Lakretz et al. (2021). *Depth generalization as a distinct failure axis*. arXiv:2101.02258. — [transformer-expressivity] depth generalization fails separately from length (PCC -0.92). [unverified — GPT-2-scale]
10. Dziri et al. (2023). *Faith and Fate: Limits of Transformers on Compositionality*. arXiv:2305.18654 (NeurIPS23). — [transformer-expressivity] transformers = linearized subgraph matching; error→1 with complexity; supports decomposition. (cross-link: §2 scale limits)
11. Jackson, Lee, Servedio & Wan (2008). *Learning random monotone DNF*. RANDOM08. — [transformer-expressivity] OR/disjunctive structure recovery is the core difficulty; classical PAC theory.
12. Zhou et al. (2023). *What Algorithms can Transformers Learn? (RASP-L length generalization)*. arXiv:2310.16028. — [transformer-expressivity] length-generalization criterion; flagged as killed/untrusted as a predictor. [unverified — claims killed in adversarial check]

13. Yehudai, Amsel & Bruna (2025). *CoT trades depth for sequential sub-result tokens*. arXiv:2503.01544 (NeurIPS25). — [transformer-expressivity] 2-layer transformer solves Boolean-formula CRQ via CoT depth $\leftrightarrow$ n exchange. [unverified — future-dated]
14. (RoPE bound authors) (2025). *Expressivity bound of RoPE transformers*. arXiv:2411.07602 (EMNLP25). — [transformer-expressivity] if $TC_0 \neq NC_1$, const-depth transformer cannot evaluate BFVP/arith-formula.
15. Malach (2019). *The role of local correlation in learning (gate-label)*. arXiv:1910.11923 (JMLR22). — [transformer-expressivity] gate-label local correlation needed for learnability.
16. Buss (1987). *The Boolean formula value problem is in ALOGTIME (BFVP NC1-complete)*. — [transformer-expressivity] BFVP=NC1-complete; theoretical ceiling for single-pass evaluation.
17. Dehghani et al. (2018). *Universal Transformers*. arXiv:1807.03819. — [transformer-expressivity] recurrence is Turing-complete (under assumptions); serial depth = iterations.
18. Giannou et al. (2023). *Looped Transformers as Programmable Computers*. arXiv:2301.13196. — [transformer-expressivity] looped 13-layer = programmable computer; constructive existence proof.
19. Fan et al. (2025). *Looping for length/depth generalization (n-RASP-L)*. arXiv:2409.15647 (ICLR25). — [transformer-expressivity] adaptive-step looping improves length/depth generalization with step supervision.
20. Geiping et al. (2025). *Huginn-3.5B: latent recurrent-depth reasoning*. arXiv:2502.05171. — [transformer-expressivity] from-scratch latent recurrence; retrofit-incompatible; latent < CoT. [unverified — future-dated]
21. Lu et al. (2025). *Probing latent CoT*. arXiv:2507.02199. — [transformer-expressivity] interpretable latent CoT nearly absent; latent $\blacksquare$ explicit CoT. [unverified — future-dated]
22. (TRM/recurrence authors) (2026). *Tiny Recursive Model / UT+ACT comparison*. arXiv:2604.21999. — [transformer-expressivity] recursive-depth TRM 87.4% vs UT+ACT 6-8% on Sudoku. [unverified — future-dated]
23. (recurrence-generalization authors) (2025). *Recurrence $\neq$ generalization alone*. arXiv:2510.04871. — [transformer-expressivity] recurrence needs correct training to generalize. [unverified — future-dated]
24. (additional depth-recurrence source) (2025). *Depth-recurrence internalization variant*. arXiv:2509.25239. — [transformer-expressivity] depth-recurrence source (primary list). [unverified — future-dated]
25. Li et al. (2024). *CoT enables serial / P-complete computation*. — [transformer-expressivity] CoT adds serial steps per token, enabling unbounded depth (P-complete); time-in-LLM payment.
26. van Benthem (1986). *Essays in Logical Semantics (semantic automata)*. — [semantic-automata] quantifiers encode automata complexity classes; "most" needs pushdown; depth measure foundation.
27. Barwise & Cooper (1981). *Generalized Quantifiers and Natural Language*. Linguistics and Philosophy. — [semantic-automata] generalized quantifier theory; basis for procedure-typed quantifier analysis.
28. Goodman (1968). *Languages of Art (notationality)*. — [procedural-semantics] notationality = disjointness + finite differentiation for unambiguous machine manipulation.
29. Blakemore / Wilson & Sperber (1987-2011). *Relevance Theory: procedural vs conceptual meaning*. — [procedural-semantics] linguistic items encode concepts (denotation) or procedures (inference routes); maps denotation vs procedure split.
30. Groenendijk, Stokhof, Veltman / Heim (1990s). *Dynamic semantics (meaning as context-change potential)*. — [procedural-semantics] meaning = information-state update instruction; denotation/procedure coexist.
31. Iverson (1979). *Notation as a Tool of Thought*. CACM (Turing Award lecture). — [procedural-semantics] notation embedding algorithm; depth-of-notation lineage. (cross-link: §12 classics)
32. Zhang & Norman (1995). *A representational analysis of numeration systems*. Cognition. — [procedural-semantics] representation sets cost profile not computability; Arabic vs Roman = internal vs external step tradeoff. (cross-link: §12 classics)
33. Bennett (1988). *Logical Depth and Physical Complexity*. — [procedural-semantics] logical depth = compute-time to unfold compact symbol; grounds d(e) notation-depth measure. (cross-link: §12 classics)
34. Wittgenstein (1953). *Philosophical Investigations (meaning as use)*. — [procedural-semantics] meaning-as-use captured by shallow association; LLM strength on use/sense. (cross-link: §12 classics)

35. Frege (1892). *Über Sinn und Bedeutung (sense vs reference)*. — [procedural-semantics] sense (grasp=association) vs reference (compute=procedure); LLM strong on sense, weak on reference-compute. (cross-link: §12 classics)
36. Brandom (1994/2000). *Material vs formal inference*. — [procedural-semantics] material (immediate) vs formal (computed) inference; LLM strong on material, stalls on formal. (cross-link: §12 classics)
37. Harris (1954) / Firth (1957). *Distributional hypothesis ("you shall know a word by the company it keeps")*. — [procedural-semantics] meaning as distributional geometry; low-dimensional manifold learnable by gradient descent.
38. Feldman (2013). *The neural binding problem(s) / ~100-step rule*. — [cognitive-architecture] brain ~100-step serial limit; language evolved for bounded-depth parallel cognition; grounds shallow-parallel thesis.
39. Clark & Chalmers (1998). *The Extended Mind*. Analysis. — [cognitive-architecture] humans offload deep procedures to external serial tools; LLM=language core, algorithm=external tool.

2. Scale limits & genuine model limits — compositional/reasoning collapse, hallucination inevitability & abstention, floor measurement, metacognition/self-evaluation

-
40. Xu, Jain & Kankanhalli (2024). *Hallucination is Inevitable: An Innate Limitation of LLMs*. arXiv:2401.11817. — [hallucination-abstention] formal (diagonalization) limit independent of scale; precision=1 unbuyable.
 41. Kalai, Nachum, Vempala & Zhang (2025). *Why Language Models Hallucinate*. arXiv:2509.04664 (OpenAI). — [hallucination-abstention] hallucination = binary-classification errors + guessing-incentives; not a scale problem.
 42. Mirzadeh et al. (2024). *GSM-Symbolic*. arXiv:2410.05229 (Apple). — [hallucination-abstention] single distractor clause drops SOTA up to 65%; no genuine logical reasoning.
 43. Valmeekam, Stechly & Kambhampati (2024). *PlanBench / o1 evaluation*. arXiv:2409.13373. — [hallucination-abstention] o1 plain 97.8% but obfuscated 52.8%, 20-40 steps 23.6%; symbolic planner 100%.
 44. (constraint-fabrication authors) (2025). *Reasoning models fabricate constraints*. arXiv:2505.12151. — [hallucination-abstention] reasoning models hallucinate absent constraints (67-94% of false errors). [unverified — future-dated]
 45. (Reject-option survey authors) (2021). *Machine Learning with a Reject Option (survey)*. arXiv:2107.11277. — [hallucination-abstention] (h,r) predictor+rejector; 3 classes separated/dependent/integrated.
 46. (Don't-Hallucinate-Abstain authors) (2024). *Don't Hallucinate, Abstain*. arXiv:2402.00367 (ACL 2024). — [hallucination-abstention] abstention externalized via multi-LLM collaboration (because self-reflection/held-out fails); supports non-introspection.
 47. Huang et al. (n.d.). *Large Language Models Cannot Self-Correct Reasoning Yet*. (ICLR). — [small-model-reasoning] internal self-correction degrades reasoning, external verify needed.
 48. Zhang et al. (2024). *Small models need strong external verifiers*. ACL 2024. — [small-model-reasoning] small models fail self-verify on memorization-heavy substeps.
 49. (Authors n/a) (2025). *T1: external-verifier scaling for small-model reasoning*. (2025). — [small-model-reasoning] small models need strong external verifier.
 50. Zhou et al. (2023). *Least-to-Most Prompting Enables Complex Reasoning in LLMs*. ICLR 2023. — [small-model-reasoning] SCAN 16%→≥99%; decomposition not scale-gated.
 51. Hsieh et al. (2023). *Distilling Step-by-Step*. ACL 2023. — [small-model-reasoning] 770M T5 beats few-shot 540B PaLM with 80% less data.
 52. Chen et al. (2022). *Program of Thoughts (PoT): Disentangling Computation from Reasoning*. arXiv:2211.12588 / TMLR 2023. — [floor-measurement] program + deterministic interpreter offload, ~12% over CoT. (cross-link: §3 offload)
 53. Joren et al. (2025). *Sufficient Context: A New Lens on Retrieval-Augmented Generation*. ICLR 2025. — [floor-measurement] context-sufficiency autorater; small=reasoning-limited, large=info-limited. (cross-link: §11 benchmarks)
 54. Traub et al. (2024). *Overcoming Common Flaws in Selective Classification Evaluation (AUGRC)*. arXiv:2407.01032. — [metacognition] generalized risk-coverage curve; measurement discipline against single-point gaming. (cross-link: §12 classics)

55. (calibration-in-agents authors) (2026). *Verbalized confidence near-random in tau2-bench*. arXiv:2602.05073. — [metacognition] verbalized confidence/NLL/entropy near-random (AUROC 0.47-0.69); confidence inflates with trajectory length. [unverified — future-dated, pilot-scale]
56. (agentic-overconfidence authors) (2026). *Agentic overconfidence*. arXiv:2602.06948. — [metacognition] reinforces verbalized overconfidence finding in agents. [unverified — future-dated]
57. (tool-miscalibration authors) (2026). *Tool-call miscalibration*. arXiv:2601.07264. — [metacognition] tool-call miscalibration corroborating non-introspection. [unverified — future-dated]
58. (When2Call authors). *When2Call: learning when to call tools*. — [metacognition] when-to-call is learned (RPO■SFT); tool-necessity linearly decodable from hidden state (AUROC 0.89-0.96). (cross-link: §5 routing)
59. (Ask-or-Assume authors) (2026). *Ask-or-Assume: underspecification detection*. arXiv:2603.26233. — [metacognition] separates underspecification detection into a distinct agent; detector is an LLM judge. [unverified — future-dated]
60. (SAGE-Agent authors) (2025). *SAGE-Agent: structured belief state + EVPI stopping*. arXiv:2511.08798. — [metacognition] non-introspective structured belief + deterministic EVPI stopping; hand-specified Bayesian, not learned. [unverified — future-dated]

3. Neurosymbolic & offload (LLM proposes / deterministic engine executes) — PAL, LLM-Modulo, formalizers, RLVR-with-checker, neurosymbolic framing

61. Gao et al. (2022). *PAL: Program-Aided Language Models*. arXiv:2211.10435 / ICML 2023. — [offload] LLM decomposes / deterministic interpreter executes; PAL+Codex beats PaLM-540B+CoT by +15% on GSM8K; small+offload>540B; our division of labor exactly. (merged dr#83 + lit#96)
62. Kambhampati et al. (2024). *LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks*. arXiv:2402.01817 (ICML24). — [neurosymbolic] LLM proposer + external model-based verifier; 1:1 with our division of labor.
63. (LRM-Modulo authors) (2024). *LRM-Modulo: o1 with external verifier*. arXiv:2410.02162. — [neurosymbolic] o1 has zero correctness guarantee; soundness from external verifier; determinism orthogonal to capability.
64. (LLM-as-formalizer authors). *LLM-as-formalizer: NL→PDDL scaling*. — [offload] NL→PDDL formalizer beats direct planner (100 blocks 100% vs 20%); but loses 15/24 on simple problems (offload not always optimal).
65. (formalizer-erosion authors) (2024). *Erosion of formalizer advantage in top reasoning models*. arXiv:2412.09879. — [neurosymbolic] formalizer edge partially eroded on simple-PDDL (o3-mini/R1); messy-NL extension open. [unverified — flagged open]
66. Marcus (2020). *The Next Decade in AI: Four Steps Towards Robust AI*. arXiv:2002.06177. — [neurosymbolic] hybrid knowledge-driven reasoning path for robust AI; anti scaling-only camp.
67. Garcez & Lamb (2020). *Neurosymbolic AI: The 3rd Wave*. arXiv:2012.05876 (AI Review 2023). — [neurosymbolic] motivation = trust/safety/interpretability/accountability; anchor for auditability.
68. Kautz (2022). *The Third AI Summer (Engelmore Lecture)*. AI Magazine 43(1):105-125. — [neurosymbolic] 6-type neurosymbolic taxonomy; positions our Neuro|Symbolic cooperative package.
69. LeCun (2022). *A Path Towards Autonomous Machine Intelligence*. OpenReview BZ5a1r-kVsf. — [neurosymbolic] modular world-model architecture; structure even in deep-learning camp.
70. Trinh et al. (2024). *Solving olympiad geometry without human demonstrations (AlphaGeometry)*. Nature 625. — [neurosymbolic] neural LM proposer + symbolic deduction engine; IMO geometry 25/30; frontier is hybrid.
71. DeepMind (2024). *AlphaProof / IMO 2024 silver*. DeepMind 2024. — [neurosymbolic] proofs RL'd over Lean formal verifier; reward = deterministic verifier (RLVR's V).
72. Peer & Stabinger (2025). *ATA: Neuro-Symbolic Autonomous Trustworthy Agents*. arXiv:2510.16381. — [neurosymbolic] LLM=NL→formal KB, symbolic engine decides; thesis-framing nearest rival; KB rebuilt per domain.
73. Evans et al. (2018). *Can Neural Networks Understand Logical Entailment? (PossibleWorldNets)*. arXiv:1802.08535 (ICLR18). — [neurosymbolic] tree-structured nets exploit logic syntax; semantic enumeration best; from-scratch only.

74. Bowman et al. (2014). *Recursive neural networks for logical semantics*. arXiv:1406.1827. — [neurosymbolic] tree-structured nets learn logical relations; not LLM/transfer.
75. Chowdhury & Caragea (2021). *Modeling Hierarchical Structures with CRvNN*. ICML21 (chowdhury21a). — [neurosymbolic] CRvNN extrapolates ListOps len≤100→900-1000 at 96-98%; recursive bias.
76. (Beam Tree RvNN authors) (2023). *Beam Tree Recursive Cells*. arXiv:2305.19999 (ICML23). — [neurosymbolic] recursive nets IID 99.4%; documents unseen arg-count (fan-in) generalization failure.
77. (RIR authors) (2023). *Recursion in Recursion*. arXiv:2311.04449 (NeurIPS23). — [neurosymbolic] recursive-structure models extrapolate depth/length where vanilla transformers fail.
78. Nye et al. (2019). *Learning to Infer Program Sketches (SketchAdapt)*. arXiv:1902.06349. — [neurosymbolic] learned sketch + symbolic hole-fill; boundary is learned (our delta axis). [unverified — PDF render failed, reconfirm]
79. Lin et al. (2022). *Description Logic TBox/ABox (terminology vs assertion separation)*. — [neurosymbolic] TBox/ABox term separation adopted as tool; their symbolic is hand-written, ours learned.
80. (Intermediate Languages authors) (2025). *Intermediate Languages Matter*. arXiv:2502.17216. — [neurosymbolic] NL→formal IR choice is first-order decision variable; single-shot ProntoQA/ProofWriter. (cross-link: §7 NL→formal)
81. Qiu, Ye, Gao, Zou, Chen, Gui, Huang, Xue, Qiu, Zhao (2025). *Blueprint First, Model Second: A Framework for Deterministic LLM Workflow*. arXiv:2508.02721. — [offload-boundary] deterministic blueprint backbone, LLM only where judgment needed. [quantified results unverified]
82. (Authors n/a) (2025). *From REST to MCP: An Empirical Study of API Wrapping and Automated Server Generation for LLM Agents (AutoMCP)*. arXiv:2507.16044. — [offload-boundary] spec→server compilation 76%→94.2% with auto-repair, cuts glue.
83. (PayPal authors) (2025). *A Declarative Language for Building And Orchestrating LLM-Powered Agent Workflows*. arXiv:2512.19769. — [cost-tradeoff] declarative DSL, 76% faster modifications, config not code. [self-reported]
84. (Authors n/a) (2025). *Agentic AI: A Comprehensive Survey of Architectures, Applications, and Future Directions*. arXiv:2510.25445. — [offload-boundary] symbolic/hybrid dominate safety-critical; generate-then-validate pattern.
85. (Authors n/a) (2020). *Robotic Process Automation — A Systematic Literature Review and Assessment Framework*. arXiv:2012.11951. — [cost-tradeoff] RPA low build cost but fragile/costly maintenance (non-vendor anchor). (cross-link: §10 cost/TCO)
86. (Authors n/a) (2026). *Automatic End-to-End Data Integration using Large Language Models*. arXiv:2603.10547. — [offload-boundary] LLM auto-generates schema/value mappings, replaces glue. [unverified, future-dated]
87. (Authors n/a) (2025). *LRASGen: LLM-based RESTful API Specification Generation*. arXiv:2504.16833. — [offload-boundary] LLM generates OpenAPI specs, cuts config-authoring cost.
88. (Authors n/a) (2024). *AI-powered software testing tools: A systematic review and empirical assessment of their features and limitations*. arXiv:2409.00411. — [offload-boundary] honest counterweight on self-healing locator limits.
89. (Authors n/a) (2026). *SAGAI-MID: A Generative AI-Driven Middleware for Dynamic Runtime Interoperability*. arXiv:2603.28731. — [offload-boundary] LLM as self-healing runtime integration layer. [unverified, future-dated]
90. (Authors n/a) (2024). *GRAM: Generative Retrieval Augmented Matching of Data Schemas*. arXiv:2406.01876. — [offload-boundary] generative schema matching ~88.7% vs 75.3% prior.
91. (Authors n/a) (n.d.). *Matchmaker: Self-Improving Large Language Model Programs for Schema Matching*. OpenReview:18E2ZooCte. — [offload-boundary] zero-shot self-improving schema matching, confidence-gated.
92. (Authors n/a) (n.d.). *A Neuro-Symbolic Framework for Deterministic Reliability in AI-Assisted Structural Engineering: The SYNAPSE Architecture*. MDPI 2075-5309/16/3/534. — [offload-boundary] deterministic safety-critical math, 94% accuracy <2s.
93. ZenML (2025). *What 1,200 Production Deployments Reveal About LLMops in 2025*. zenml.io blog. — [offload-boundary] safety logic moved out of prompts into infrastructure.
94. Anthropic (n.d.). *Building Effective AI Agents*. anthropic.com/research. — [offload-boundary] deterministic workflows vs flexible agents, prefer simplest.

95. Stack Overflow Blog (2025). *Reliability for Unreliable LLMs*. blog (2025-06-30). — [offload-boundary] determinism from acceptance/validation; gate state/money/trust.
96. Google Developers Blog (n.d.). *Production-Ready AI Agents: 5 Lessons from Refactoring a Monolith*. blog. — [formalize-granularity] shift NL contract to runtime-validated typed object.
97. Google ADK (n.d.). *OpenAPI tools*. google.github.io/adk-docs. — [offload-boundary] auto-generate tools from OpenAPI; endpoint change = spec edit.
98. Microsoft Learn (n.d.). *Declarative agent architecture (Copilot extensibility)*. MS Learn docs. — [cost-tradeoff] config-not-code; OpenAPI-bound, no multistep loops.
99. Microsoft Azure AI Foundry (n.d.). *Three Tiers of Agentic AI — and When to Use None of Them*. vendor. — [offload-boundary] deterministic supervisor + agentic specialists.
100. deepset (n.d.). *AI Agents and Deterministic Workflows: A Spectrum, Not a Binary Choice*. vendor. — [offload-boundary] partition by criticality, not task sophistication.
101. Deepchecks (n.d.). *How Prompt Updates Drive Most Incidents*. vendor. — [cost-tradeoff] prompt surface itself a high change-cost liability.
102. Praetorian (n.d.). *Deterministic AI Orchestration: A Platform Architecture for Autonomous Development*. blog. — [offload-boundary] mission-critical needs deterministic, auditable execution.
103. samchon (n.d.). *samchon/openapi — OpenAPI converters + LLM function-calling schema composer*. GitHub. — [formalize-granularity] validation feedback raises tool-call success 70%→98%.

4. Cognitive architectures × LLM — SOAR, CoALA, NL2GenSym, Bootstrapping, LLM-ACTR, MERLIN2

104. (NL2GenSym authors) (2025). *NL2GenSym: NL→SOAR production via LLM*. arXiv:2510.09355. — [cognitive-architecture] LLM generates NL→SOAR rules, execution-grounded Generator-Critic; nearest neighbor; small+framework>large.
105. Wray, Kirk & Laird (2025). *Applying Cognitive Design Patterns to General LLM Agents*. arXiv:2505.07087 (AGI 2025). — [cognitive-architecture] impasse/subgoal, propose-select-reconsider as canonical; LLM self-reflection distrust; authority for SOAR framing.
106. Summers, Yao, Narasimhan & Griffiths (2024). *Cognitive Architectures for Language Agents (CoALA)*. arXiv:2309.02427 (TMLR). — [cognitive-architecture] umbrella framework; LLM=probabilistic production; memory/action/decision axes; we are an instance.
107. Wu, Oltramari, Francis, Giles & Ritter (2024). *Cognitive LLMs / LLM-ACTR*. arXiv:2408.09176. — [cognitive-architecture] ACT-R policy baked into LoRA weights; opposite direction; no transfer/abstain.
108. Zhu & Simmons (2024). *Bootstrapping Cognitive Agents with a Large Language Model*. arXiv:2403.00810 (AAAI 2024). — [cognitive-architecture] LLM bootstraps SOAR-like productions, symbolic verifies; deterministic-first/LLM-fallback, 50-100× token reduction.
109. González-Santamarta et al. (2023). *LLM in Cognitive Architecture (MERLIN2 robot)*. arXiv:2309.14945. — [cognitive-architecture] GBNF grammar-constrained decoding NL→PDDL, symbolic planner verifies; LLM=generate/symbolic=execute.
110. (NL2CA authors) (2025). *NL2CA: NL→LTL→ACT-R production compilation*. arXiv:2512.18189. — [cognitive-architecture] fine-tuned Qwen3-0.6B NL→LTL→Critic Tree→pyactr production, fully automatic; direct prior for NL→A2 generator. [unverified — future-dated] (cross-link: §7 NL→formal)
111. Jones, Wray & Laird (2026). *No-compliant-action as structural event (AAAI-26)*. AAAI 2026 (paper 41081). — [cognitive-architecture] structural no-compliant-action event; flagged as verification-failed, close-read needed. [unverified — verification failed in adversarial check]

5. Learned routing, deferral & small-model orchestration (cost) — ToolOrchestra, TRUST, RTR/A2FM/xRouter/When2Call/RITE, cascading/deferral (ReDAct, Unified Routing/Cascading)

112. (ToolOrchestra authors) (2025). *ToolOrchestra*. arXiv:2511.21689. — [cost-routing] 8B RL: tau2 80.2%@10.3¢ > GPT-5 77.7%@31.3¢; most dangerous rival, preempts "small>large" headline. [unverified — future-dated]
113. (TRUST authors) (2026). *TRUST: 4-way Direct/Tool/Ask/Unable routing*. arXiv:2606.06976. — [cost-routing] 4B>30B, Claude-Sonnet-4 parity; pure RL+perplexity, introspective; small>large co-preemptor, differentiate by method. [unverified — future-dated]
114. (RTR authors). *RTR: joint model+strategy routing*. — [cost-routing] learned joint routing with OOD transfer, 71.7% token reduction; learned routing is established.
115. (A2FM authors). *A2FM: instant/reasoning/agent mode RL routing*. — [cost-routing] RL routing across modes, 45% cost reduction; learned routing established.
116. (xRouter authors). *xRouter: answer-vs-delegate RL routing*. — [cost-routing] RL answer-vs-delegate routing; learned routing established.
117. (ARM2 authors). *ARM2: NL/code/vision format RL selection*. — [cost-routing] RL format selection with code offload; learned routing established.
118. (ToolkenGPT authors). *ToolkenGPT: tool calls as token prediction*. — [cost-routing] tool invocation as token prediction; learned routing established.
119. (RITE authors). *RITE: math-only RL training transfers cross-domain SOTA*. — [cost-routing] decomposition/tool-routing is a learned, transferable skill; supports our C8 transfer claim.
120. (TGRL authors) (2025). *TGRL: domain-invariant planning via RL*. arXiv:2510.11184. — [cost-routing] tool-use habit transfer via opaque RL; no basis/closure claim. (cross-link: §9 transfer)
121. (ReDAct authors) (2026). *ReDAct: Uncertainty-Aware Deferral for LLM Agents*. arXiv:2604.07036. — [cost-routing] calibrated-threshold defer small→large on ALFWorld/MiniGrid. [unverified — future-dated]
122. (Routing-Cascading authors) (2024). *Unified Routing and Cascading for LLMs*. arXiv:2410.10347. — [cost-routing] cascade routing; learned quality estimator is the key factor; model-selection deferral.
123. (cascade-error authors) (2025). *Modular agent cascade errors (quantified)*. arXiv:2503.13657. — [cost-routing] modular-agent cascade errors are real and quantified.
124. (Self-Healing authors) (2026). *Self-Healing Agentic Orchestrators*. arXiv:2606.01416. — [cost-routing] deterministic monitor→diagnose→recover→verify loop; fault-injection 98.8%. [unverified — future-dated]
125. (AgentDebug authors) (2025). *AgentDebug / Where LLM Agents Fail*. arXiv:2509.25370. — [cost-routing] AgentErrorTaxonomy + attribution/repair; +24%/+17%/up to 26% on ReAct benchmarks.
126. Shinn et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. arXiv:2303.11366 / NeurIPS 2023. — [learned-routing] external episodic memory, frozen policy, fast trial gains.
127. Zhao et al. (2024). *ExpeL: LLM Agents Are Experiential Learners*. arXiv:2308.10144 / AAAI 2024. — [learned-routing] distills traces into NL insights, cross-task transfer.
128. Wang et al. (2023). *Voyager: An Open-Ended Embodied Agent with LLMs*. arXiv:2305.16291. — [learned-routing] skill library indexed by NL, frozen GPT-4, compose skills; open (infinite, domain-specific) skill library = structural opposite of our closed basis. (merged dr#38 + lit#112)
129. Wang, Mao, Fried, Neubig (2024). *Agent Workflow Memory (AWM)*. arXiv:2409.07429. — [learned-routing] induces reusable abstract workflows, +24.6%/+51.1% relative.
130. (Authors n/a) (2025). *Adaptation of Agentic AI: A Survey of Post-Training, Memory, and Skills*. arXiv:2512.16301. — [learned-routing] parametric vs external memory vs skill libraries. [unverified, future-dated]
131. (Authors n/a) (2026). *Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers*. arXiv:2603.07670. — [learned-routing] distilled abstract skills > raw retrieval banks. [unverified, future-dated]
132. (Authors n/a) (2025). *A Benchmark for Procedural Memory Retrieval in Language Agents*. arXiv:2511.21730. — [learned-routing] evaluates retrieving past procedures from experience DB. [unverified, future-dated]

133. (Authors n/a) (2024). *Enhancing Decision-Making for LLM Agents via Step-Level Q-Value Models*. arXiv:2409.09345. — [learned-routing] external Q-store guides selection without retraining policy.
134. (Authors n/a) (2025). *DG-PRM: Dynamic and Generalizable Process Reward Modeling*. arXiv:2507.17849 / ACL 2025. — [learned-routing] external reward-tree dynamically selected, OOD-generalizable; swappable store.
135. (Authors n/a) (2026). *ToolPRMBench: Evaluating and Advancing PRMs for Tool-using Agents*. arXiv:2601.12294. — [learned-routing] extends step-PRMs to tool-use, checklist sub-goal rewards. [unverified, future-dated]
136. Arumugam et al. (2025). *Toward Efficient Exploration by Large Language Model Agents*. arXiv:2504.20997. — [learned-routing] PSRL via LLMs, retains provable Bayesian regret guarantees.
137. Xia et al. (2024). *Beyond Numeric Rewards: In-Context Dueling Bandits with LLM Agents (LEAD)*. arXiv:2407.01887. — [learned-routing] LEAD inherits regret guarantees from external bandit, not LLM.
138. (Authors n/a) (2024). *Which LLM to Play? Convergence-Aware Online Model Selection with Time-Increasing Bandits (TI-UCB)*. arXiv:2403.07213. — [learned-routing] non-stationary success-rate prioritizer with regret analysis.
139. (Authors n/a) (2025). *A Multi-Agent Conversational Bandit Approach to Online Evaluation and Selection of User-Aligned LLM Responses*. arXiv:2501.01849. — [learned-routing] provably-improving external selector over LLM candidates.
140. (Authors n/a) (2025). *QLASS: Boosting Language Agent Inference via Q-Guided Stepwise Search*. arXiv:2502.02584. — [learned-routing] Q-guided stepwise search (raw hit). [unverified]
141. (Authors n/a) (2025). *AgentPRM: Process Reward Models for LLM Agents via Step-Wise Promise and Progress*. arXiv:2511.08325. — [learned-routing] PRM capturing decision interdependence (raw hit). [unverified, future-dated]
142. (Authors n/a) (2025). *TDRM: Smooth Reward Models with Temporal Difference*. arXiv:2509.15110. — [learned-routing] smoothed reward models (raw hit). [unverified]
143. (Authors n/a) (n.d.). *Principle Process Reward (PPR)*. OpenReview (no arXiv id). — [learned-routing] unifies step assessment + outcome verification (raw hit). [unverified]
144. (Authors n/a) (2025). *Agent KB: Leveraging Cross-Domain Experience for Agentic Problem Solving*. arXiv:2507.06229. — [learned-routing] cross-domain experience reuse (raw hit). [unverified]
145. (Authors n/a) (2026). *ProcMEM: Learning Reusable Procedural Memory via Non-Parametric PPO*. arXiv:2602.01869. — [learned-routing] reusable procedural memory (raw hit). [unverified, future-dated]
146. (Authors n/a) (2025). *LEGOMem: Modular Procedural Memory for Multi-agent LLM Systems*. arXiv:2510.04851. — [learned-routing] modular procedural memory (raw hit). [unverified]
147. (Authors n/a) (2025). *How Memory Management Impacts LLM Agents: experience-following behavior*. arXiv:2505.16067. — [learned-routing] memory management / experience-following (raw hit). [unverified]

6. Guarantees, shielding & safety enforcement — safe-RL shielding, CBF/safety-filters, ShieldAgent, AgentSpec, Formal-LLM, solver-aided policy compliance

148. Alshiekh et al. (2018). *Safe Reinforcement Learning via Shielding*. AAAI 2018. — [shielding-safety] LTL spec→deterministic automaton→safety game→shield; archetype of propose-then-shield.
149. Jansen et al. (2018). *Safe RL via Probabilistic Shields*. arXiv:1807.06096. — [shielding-safety] probabilistic relaxation of shielding; variant lineage.
150. Wabersich & Zeilinger (2021). *Predictive safety filter (pCBF)*. Automatica 2021; arXiv:2105.10241. — [shielding-safety] modular safety filter appended to any learned controller; proposer-independent.
151. Hewing/Wabersich et al. (2023). *The Safety Filter: A Unified View*. Annual Reviews 2023. — [shielding-safety] CBF/predictive-filter/HJ-reachability unified survey; paradigm maturity.
152. (ShieldAgent authors) (2025). *ShieldAgent: guardrail agent from policy rules*. arXiv:2503.22738. — [shielding-safety] policy→verifiable rules→probabilistic rule circuits; nearest neighbor, probabilistic not deterministic. [unverified — abs read only]

153. (AgentSpec authors) (2025). *AgentSpec: runtime constraint DSL*. arXiv:2503.18666. — [shielding-safety] trigger/predicate/enforcement DSL, deterministic but hand-written per-domain spec. (cross-link: §7 NL→formal)
154. (Formal-LLM authors) (2024). *Formal-LLM: automaton-constrained generation*. arXiv:2402.00798. — [shielding-safety] developer-written automata supervise generation; generation-constraint variant, no transfer.
155. (ProbGuard authors) (2025). *ProbGuard: probabilistic runtime monitoring*. arXiv:2508.00500. — [shielding-safety] probabilistic runtime monitoring; one-line variant. [unverified — abs only]
156. Winston, Winston & Just (2026). *Solver-Aided Verification of Policy Compliance*. arXiv:2603.20449. — [shielding-safety] NL policy→SMT-LIB→Z3 runtime gate on tau-bench; gate-leg nearest rival; human-translated, airline only. [unverified — future-dated]

7. NL→formal / spec generation & constrained decoding — NL2CA, Prose2Policy, AgentSpec, StepFun-Formalizer, FormalAlign, VeriEquivBench, intermediate-languages, CRANE / Let-Me-Speak-Freely

157. Guo et al. / IRNet (2019). *Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation (IRNet/SemQL)*. arXiv:1905.08205 / P19-1444. — [formalize-granularity] schema-link→SemQL→deterministic SQL; reference-emit, cross-domain transfer.
158. Dong & Lapata (2018). *Coarse-to-Fine Decoding for Neural Semantic Parsing*. ACL P18-1068. — [formalize-granularity] value-free sketch first then fill, shared across examples.
159. (Authors n/a) (2019). *Sketch-based semantic parsing*. arXiv:1909.00574. — [formalize-granularity] sketch-then-fill; value-filling is itself learned.
160. (Authors n/a) (2021). *PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding*. EMNLP 2021.emnlp-main.779. — [constrained-decoding] T5-3B 68-71%→75.1%, execution error 12%→2%.
161. (Authors n/a) (2020). *RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL*. arXiv:1911.04942. — [formalize-granularity] schema-as-input relation-aware self-attention, zero-shot schema transfer.
162. (Authors n/a) (2017). *Execution-Guided Decoding for Text-to-SQL*. arXiv:1807.03100. — [constrained-decoding] deterministic execution engine filters faulty programs, WikiSQL 83.8%.
163. (Authors n/a) (2025). *Grammar-Constrained Decoding as Parser (form-not-meaning)*. ACL 2025.acl-industry.34. — [constrained-decoding] grammar guarantees form not meaning; CFG ceiling, needs solver. [semantic-improvement claim REFUTED]
164. Tam et al. (2024). *Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of LLMs*. arXiv:2408.02442 / EMNLP 2024 Industry. — [constrained-decoding] schema-JSON drops GSM8K ~26–73%; two-stage recovers.
165. (Authors n/a) (2025). *CRANE: Reasoning with Constrained LLM Generation*. arXiv:2502.09061 / ICML 2025. — [constrained-decoding] grammar augmentation + alternating constrained/free, +10pp GSM-sym/FOLIO.
166. Ye et al. (2024). *Guided Decoding distribution distortion (GAD)*. NeurIPS 2024. — [constrained-decoding] greedy token-masking + renormalization distorts distribution (KL-bias).
167. (Authors n/a) (n.d.). *JSONSchemaBench (in-schema reasoning-field-first)*. (benchmark). — [constrained-decoding] free-text reasoning field inside schema first recovers reasoning.
168. Kim, Poiroux & Bosselut (2026). *Do LLMs Game Formalization?*. arXiv:2604.19459 (EPFL, ICLR-2026 VerifAI workshop). — [hallucination-abstention] NL→Lean4: R1 premise-mistranslation undetectable, GPT-5 axiom-fabrication detectable; supports silent-residual delta. [unverified — future-dated]

8. Search, planning, tree-evaluation & process-reward internalization — ToT/RAP/LATS/ToolChain*, Searchformer/TS-LLM, Math-Shepherd/Let's-Verify/ReST-MCTS/OmegaPRM, loop-vs-CoT, tree-eval learnability, path/plan selection

169. Yao et al. (2023). *Tree of Thoughts (ToT)*. arXiv:2305.10601 (NeurIPS23). — [search-internalization] external tree search; CoT 4%→74%; analogical motivation (we remove external search).
170. Hao et al. (2023). *Reasoning with Language Model is Planning with World Model (RAP)*. arXiv:2305.14992 / EMNLP 2023. — [search-internalization] LLM as world model + policy, process-reward MCTS; LLaMA-33B+RAP > GPT4-CoT +33%; external search to be internalized. (merged dr#32 + lit#26)
171. Zhou et al. (2024). *Language Agent Tree Search (LATS)*. arXiv:2310.04406 / ICML 2024. — [search-internalization] external MCTS controller + LLM self-eval value, no weight updates; 92.7% HumanEval; analogical. (merged dr#31 + lit#27)
172. Besta et al. (2024). *Graph of Thoughts (GoT)*. AAAI24 (29720). — [search-internalization] graph beats tree on accuracy and cost (>31% reduction); analogical motivation.
173. (DPTS authors) (2025). *Dynamic Parallel Tree Search (DPTS)*. arXiv:2502.16235 (ACL25). — [search-internalization] external tree-search efficiency; analogical. [unverified — future-dated]
174. Zhuang et al. (2024). *ToolChain*: Efficient Action Space Navigation with A* Search*. arXiv:2310.13227 / ICLR 2024. — [search-internalization] external A* cost/heuristic over API-call decision tree.
175. (Authors n/a) (2025). *Unifying Tree Search Algorithm and Reward Design for LLM Reasoning: A Survey*. arXiv:2510.09988. — [search-internalization] taxonomizes tree-search families vs reward designs.
176. (Authors n/a) (2024). *Scaling of Search and Learning: A Roadmap to Reproduce o1*. arXiv:2412.14135. — [search-internalization] search+learning roadmap (raw hit). [unverified]
177. (Authors n/a) (2026). *ToolTree: Dual-Feedback MCTS + Bidirectional Pruning*. arXiv:2603.12740. — [search-internalization] tool-planning tree search (raw hit). [unverified, future-dated]
178. Lehnert et al. (2024). *Searchformer: Beyond A with search-dynamics bootstrapping*. arXiv:2402.14083 (TMLR, Meta). — [search-internalization] keystone: A trace distillation + shorter-trace bootstrapping; teacher-exceeding generalization.
179. (Searchformer-critique authors) (2025). *Critique of Searchformer mechanism*. arXiv:2505.13775. — [search-internalization] critiques the mechanism (not the numbers) of trace-distillation. [unverified — future-dated]
180. Feng et al. (2024). *TS-LLM: Tree-Search with learned value function*. arXiv:2309.17179 (ICML24). — [search-internalization] learned value-function distillation reaches depth-64; adaptation for depth risk.
181. Zhang et al. (2024). *ReST-MCTS*: LLM Self-Training via Process Reward Guided Tree Search*. arXiv:2406.03816 (NeurIPS24). — [search-internalization] external search guidance plus per-step process reward auto-inferred from final answer; internalization via self-training; recipe for our verifier. (merged dr#35 + lit#33)
182. Wang et al. (2024). *Math-Shepherd: Verify and Reinforce LLMs Step-by-step without Human Annotations*. arXiv:2312.08935 / ACL 2024. — [small-model-reasoning] auto step-PRM, Mistral-7B GSM8K 89.1%; step labels from final gate-answer; external + internalize modes. (merged dr#47 + lit#34)
183. Lightman et al. (2023). *Let's Verify Step by Step*. arXiv:2305.20050 / OpenAI; ICLR 2024. — [small-model-reasoning] PRM>ORM, MATH 78%, PRM800K; external verifier ranks candidates.
184. Luo et al. (2024). *OmegaPRM / Improve Mathematical Reasoning by Automated Process Supervision*. arXiv:2406.06592 / Google DeepMind. — [small-model-reasoning] MCTS+binary search locates first error, scales PRM labels.
185. Tian et al. (2024). *AlphaLLM (imagination-searching-criticizing)*. arXiv:2404.12253 (NeurIPS24). — [search-internalization] search-trace self-train > ReST^EM/Self-Rewarding; expert-iteration recipe.
186. Guan et al. (2025). *rStar-Math: 7B self-evolution via MCTS*. arXiv:2501.04519 (ICML25). — [search-internalization] 7B self-evolution without superior teacher; 58.8→90 MATH number is killed/cite-forbidden. [unverified — specific number killed]
187. Song et al. (2024). *Trial and Error: Exploration-Based Trajectory Optimization (ETO)*. arXiv:2403.02502 / ACL 2024. — [search-internalization] whole-trajectory DPO internalizes success>failure into weights.
188. Putta et al. (2024). *Agent Q: Advanced Reasoning and Learning for Autonomous AI Agents*. arXiv:2408.07199. — [search-internalization] MCTS + self-critique + off-policy DPO. [figures unverified]
189. (additional internalization source) (2024). *Self-train / expert-iteration variant*. arXiv:2404.03683. — [search-internalization] internalization recipe (primary source list).

190. (additional internalization source) (2024). *Trace/value internalization variant*. arXiv:2405.14838. — [search-internalization] internalization recipe (primary source list).
191. (additional internalization source) (2024). *Trace/search internalization variant*. arXiv:2407.06023. — [search-internalization] internalization recipe (primary source list).
192. Xu & Sato (2025). *To CoT or To Loop? A formal comparison*. arXiv:2505.19245. — [transformer-expressivity] deterministic DAG/tree evaluation: loop $\propto$ depth, CoT $\propto$ size (formal separation). [unverified — future-dated]
193. (RELAY authors) (2025). *RELAY: loop-iteration $\leftrightarrow$ CoT-step alignment*. arXiv:2502.08482. — [search-internalization] recurrence as trace generator; distill looped traces into AR model via SFT. [unverified — future-dated]
194. (Authors n/a) (2025). *A Survey of Process Reward Models: From Outcome Signals to Process Supervisions*. arXiv:2510.08049. — [small-model-reasoning] PRM survey (raw hit). [unverified]
195. (Authors n/a) (n.d.). *Toward Large Reasoning Models: reinforced reasoning with LLMs*. PMC12546433. — [small-model-reasoning] reasoning-model survey (secondary). [unverified]

9. Transfer — schema-guided dialogue & tool function-calling — SGD, D3ST, STAR, Description-Driven, ToolLLM, Tool-Doc, TGRL

196. Rastogi et al. (2020). *Schema-Guided Dialogue (SGD)*. — [benchmarks] finite acts + schema for unseen-service transfer; DST not tool-use planning; no closure proof.
197. (D3ST authors) (2022). *D3ST / Description-Driven TOD*. arXiv:2201.08904. — [benchmarks] schema reading (NL description $\rightarrow$ slot); pure neural DST; no closure basis.
198. (STAR authors) (2020). *STAR: schema-guided task-oriented dialogue with flowcharts*. arXiv:2010.11853. — [benchmarks] finite acts, explicitly non-closed; flowchart=data; exact opposite of our rule-as-abstraction.
199. (ToolLLM authors) (2023). *ToolLLM: Facilitating LLMs to Master 16000+ Real-World APIs*. arXiv:2307.16789. — [benchmarks] open tool set (16k, infinite); cross-API OOD; pure LLM; NL $\rightarrow$ API tool-plan setting; preempts cross-bench transfer. (merged dr#75 + lit#128)
200. (Tool-Doc authors) (2023). *Tool documentation enables zero-shot tool usage*. arXiv:2308.00675. — [benchmarks] tool-doc-driven tool use; FC transfer mechanism.
201. (Schema Augmentation authors) (2024). *Schema Augmentation for zero-shot transfer*. arXiv:2411.00150. — [benchmarks] schema augmentation for unseen-domain transfer; concrete prior for ABox-swap.
202. Patil et al. (2023). *Gorilla: Large Language Model Connected with Massive APIs*. arXiv:2305.15334. — [formalize-granularity] NL $\rightarrow$ API calling setting (sourced, verification claim did not survive).
203. Li et al. (2023). *API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs*. arXiv:2304.08244. — [formalize-granularity] NL $\rightarrow$ API benchmark (sourced, verification claim did not survive).

10. Cost / TCO / regulatory / scaling stance — deterministic-vs-learned TCO, Sutton/Brooks/Chollet, Palantir, EU AI Act / SR 11-7 / SR 26-2 / EU MDR / MiFID, GDPR

204. Sutton (2019). *The Bitter Lesson*. incompleteideas.net/IncIdeas/BitterLesson.html. — [bitter-lesson] general methods leveraging computation win; search AND learning; targets injected know-how, not spec-enforcement.
205. Brooks (2019). *A Better Lesson*. rodneymore.com/a-better-lesson/. — [bitter-lesson] human architecture design relocated not removed; total-cost framing = our small+gate leg.
206. Chollet (2019). *On the Measure of Intelligence*. arXiv:1911.01547. — [bitter-lesson] unlimited priors/data "buy" skill, masking generalization; justifies zero-retrain transfer measurement.
207. Karpathy (2025). *On Sutton's Bitter Lesson (X / interview reaction)*. X, 2025-10. — [bitter-lesson] bitter lesson as "biblical text" in frontier circles; non-academic, contested. [unverified — non-academic]
208. Sutton & Patel (2025). *Richard Sutton interview ("LLMs are a dead end") + follow-up*. Dwarkesh Patel podcast 2025-09. — [bitter-lesson] Sutton: LLMs inject human knowledge, not bitter-lesson-pilled; undercuts attack's minor premise.

209. European Union (2024). *EU AI Act, Regulation (EU) 2024/1689*. EUR-Lex ELI reg/2024/1689. — [regulatory-determinism] mandates traceability/logging/oversight; "deterministic"=0 occurrences; Art.15(4) presumes adaptive systems.
210. Federal Reserve (2011). *SR 11-7: Guidance on Model Risk Management*. Fed SR Letter 2011. — [regulatory-determinism] presumes statistical/uncertain models; requires validation/effective-challenge, not determinism.
211. Fed/OCC/FDIC (2026). *SR 26-2: Model Risk Management (supersedes SR 11-7/21-8)*. Fed SR Letter SR2602; OCC Bulletin 2026-13. — [regulatory-determinism] excludes deterministic rule-based processes from "model"; genAI/agent out of scope.
212. European Union (2017). *EU MDR 2017/745, Annex I §17.1*. EUR-Lex CELEX:32017R0745. — [regulatory-determinism] only explicit "repeatability" clause; medical-device, "in line with intended use" qualified.
213. FDA (2019). *AI/ML-Based SaMD Discussion Paper ("locked algorithm")*. FDA 2019-04. — [regulatory-determinism] historical "locked algorithm" definition; nonbinding; trajectory moving away from locked.
214. FDA/Health Canada/MHRA (2021). *Good Machine Learning Practice (GMLP) 10 Guiding Principles*. gov.uk MHRA publication. — [regulatory-determinism] "deterministic/reproducible/repeatable"=0; robustness + real-world monitoring.
215. European Union (2016). *GDPR Article 22 (automated decision-making)*. gdpr-info.eu. — [regulatory-determinism] safeguards = human intervention + contest; no determinism requirement.
216. European Commission (2017). *MiFID II RTS 6, Commission Delegated Regulation (EU) 2017/589*. EUR-Lex CELEX:32017R0589. — [regulatory-determinism] behave-as-intended testing; no determinism wording, (b)-light burden. [unverified — snippet, reconfirm before citing]
217. Snell et al. (2024). *Scaling LLM Test-Time Compute Optimally....* arXiv:2408.03314. — [cost-tradeoff] external search-vs-verifier beats bigger policy at fixed compute, 14× larger.
218. Ampersand (n.d.). *The Integration Debt Trap: Why Building Integrations In-House Breaks Down at Scale*. withampersand.com. — [cost-tradeoff] 160–480 hr/yr, \$50k–\$150k/yr per integration.
219. Airbyte (n.d.). *How to Build API Integrations That Don't Break*. airbyte.com. — [cost-tradeoff] one provider change propagates across all tenants.
220. Duvo (n.d.). *Why Every RPA Project Breaks (And How Agentic AI Fixes It)*. blog.duvo.ai. — [cost-tradeoff] 30–50% RPA failure, presentation-layer coupling.
221. Truto (n.d.). *Building Integrations In-House and Other Horror Stories*. vendor. — [cost-tradeoff] integrations silently stop syncing until customer notices.
222. denlava / DEV Community (2023). *Reducing API Test Brittleness: Strategies Against Minor Schema Changes*. DEV (2023-04-03). — [cost-tradeoff] hardcoded fields +70% test maintenance; schema-aware cuts 50–70%.
223. Ampersand / Towards Data Science (n.d.). *From Vibe Coding to Spec-Driven Development*. TDS. — [cost-tradeoff] spec as source of truth makes approach robust.
224. (Author n/a) (2026). *Spec-driven development case study with Claude Code*. blog (2026-05-12). — [cost-tradeoff] spec authority discipline. [partial identifier]

11. Benchmarks — tau2, SOPBench, SOP-Bench, TaskBench, CFB, AppWorld, TravelPlanner, NESTful, NATURAL PLAN, Sufficient Context

-
225. Nangia & Bowman (2018). *ListOps: A Diagnostic Dataset for Latent Tree Learning*. arXiv:1804.06028 (NAACL18). — [benchmarks] nested-operator evaluation diagnostic; latent-tree structure discovery fails, not capacity.
 226. Tay et al. (2021). *Long Range Arena (LRA)*. arXiv:2011.04006 (ICLR21). — [benchmarks] long-range benchmark including ListOps; structure-discovery scope.
 227. (ORCHARD authors) (2021). *ORCHARD: nested-operator evaluation*. arXiv:2111.14034. — [benchmarks] nested-operator scope diagnostic; scope-only relevance.

12. Classics & adopted tools — Codd, Böhm-Jacopini, Libkin-Wong, Goodman, van Benthem, Wittgenstein/Frege/Brandom, Iverson, Bennett, Zhang-Norman, Koriat/Fleming/Rouy, Traub AUGRC, Tulu 3, InstructGPT/DPO/STaR/RAG

228. Codd (1970/1972). *Relational completeness of data base sublanguages (relational algebra)*. — [procedural-semantics] relational algebra as closed finite content-operation basis; data-flow generators.

229. Böhm & Jacopini (1966). *Flow diagrams, Turing machines and languages with only two formation rules*. CACM. — [procedural-semantics] structured-program theorem; closed finite flow basis (P1-P9) for tool-use planning.

Unverified / to-recheck before submission

The following entries carry an [unverified], future-dated, (Authors n/a), partial-identifier, or verification-failed flag and must be re-checked before submission:

- §1: 8, 9, 12, 13, 20, 21, 22, 23, 24
- §2: 44, 55, 56, 57, 59, 60
- §3: 65, 78, 81, 82 (Authors n/a), 83, 84 (Authors n/a), 85 (Authors n/a), 86, 87 (Authors n/a), 88 (Authors n/a), 89, 90 (Authors n/a), 91 (Authors n/a), 92 (Authors n/a)
- §4: 110, 111
- §5: 112, 113, 121, 124, 130 (Authors n/a), 131 (Authors n/a), 132 (Authors n/a), 133 (Authors n/a), 134 (Authors n/a), 135 (Authors n/a), 138 (Authors n/a), 139 (Authors n/a), 140 (Authors n/a), 141 (Authors n/a), 142 (Authors n/a), 143 (Authors n/a), 144 (Authors n/a), 145 (Authors n/a), 146 (Authors n/a), 147 (Authors n/a)
- §6: 152, 155, 156
- §7: 159 (Authors n/a), 160 (Authors n/a), 161 (Authors n/a), 162 (Authors n/a), 163 (Authors n/a), 165 (Authors n/a), 167 (Authors n/a), 168
- §8: 173, 176 (Authors n/a), 177, 179, 186, 188, 192, 193, 194 (Authors n/a), 195 (Authors n/a)
- §10: 207, 216, 224 (partial identifier)

Note: many (Authors n/a) / (Author n/a) entries reflect placeholder author fields carried verbatim from the source lists; these need an authoritative author attribution before final citation. All future-dated arXiv ids (2025–2606) are flagged as [unverified] per source.